Adaptive Communication Network with Failure Recovery

SDN + UDP Socket Programming (Jackfruit Mini Project, Problem 30)

Project Report: Deliverable 1 and Deliverable 2 | Computer Networks (UE25CS243A)

Field	Details
Student name(s)	MUNNANGI VIJAYA ADITHYA NIRANJAN BASAPPA BENAKANAVAR PRAJWAL MANJUNATH MALLAPPANAVAR
SRN / Roll number(s)	PES1UG25CS317 PES1UG25CS343 PES1UG25CS370
Section / Faculty	F section / RADHIKA M HIRANNAIAH
GitHub repository	https://github.com/dotpmm/Adaptive-UDP-communication-with-SDN-based-failure-recovery
Tools	Ryu 4.34 (OpenFlow 1.3), Mininet 2.3.0, Open vSwitch 3.7.1, Python 3, VirtualBox Ubuntu VM

1. Summary

This report describes the complete project: the design and implementation of a resilient UDP application running over an SDN-controlled network (Sections 2 to 6), and its systematic evaluation against a static-routing baseline (Sections 7 to 12). When a network link fails, an OpenFlow controller detects the failure and reroutes the application's traffic over an alternate path, and the application itself detects the outage and resends lost data.

Main results (single link failure, 30 s run, 2 clients, mean of 3 runs):

Setup	Service outage	Messages lost	p95 latency
Static routing + plain UDP (baseline)	12.0 s	40.1 %	33 ms
Static routing + app recovery	12.1 s	0.0 %	10.6 s
SDN reroute + plain UDP	41 ms	0.14 %	42 ms
SDN reroute + app recovery (full system)	**70 ms**	**0.0 %**	**56 ms**

- SDN rerouting shortens the service outage from about 12 seconds (the whole time the link was down) to tens of milliseconds.
- Application-level recovery removes the remaining message loss. Without a working network path it turns loss into delay, which is why it is combined with SDN rerouting.
- When both paths are down (double failure), no routing can help; the application still delivers every message once a path returns.
- All numbers come from only 3 repetitions on one emulated VM, so differences of tens of milliseconds are within the noise (Section 12).

2. Problem statement and objectives

Networks fail: links go down, and a plain UDP application gives no delivery guarantee when they do. This project builds a resilient UDP communication framework in which an SDN controller

detects network failures and dynamically reroutes the application's traffic through an alternate path, while the application itself detects the outage and recovers lost data.

Objectives (from the problem statement):

- Implement UDP communication using low-level sockets.
- Create multiple network paths between the communicating hosts.
- Monitor topology and link status.
- Simulate link/path failures.
- Detect failures dynamically and reroute application traffic.
- Implement application-level recovery.
- Measure recovery time, packet loss and latency.

3. System architecture

Two UDP clients (h1, h3) send messages to a UDP server (h2) across five OpenFlow switches. Two disjoint paths exist: a short primary path through s2 and a longer, slower backup path through s3 and s5. A Ryu controller is connected to every switch.

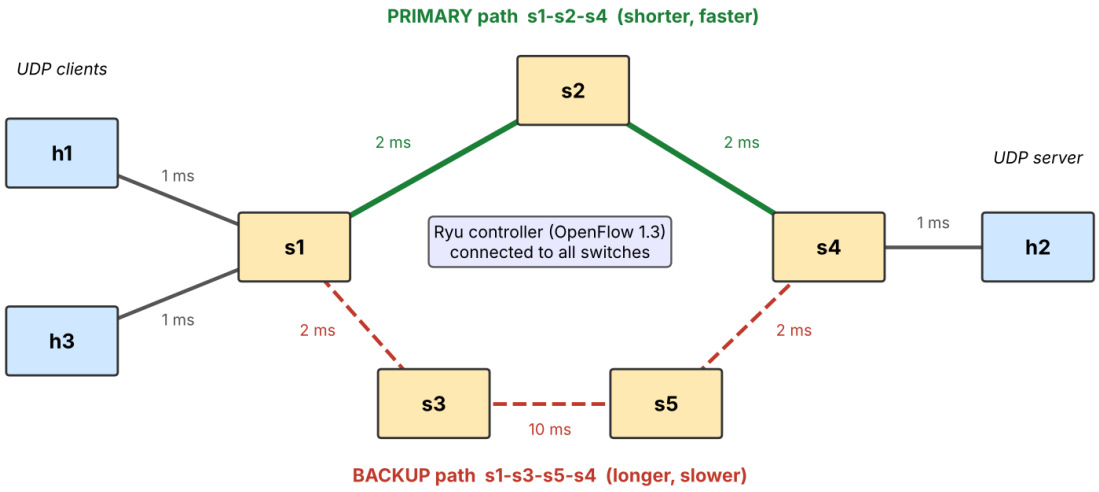


Figure 1. Mininet topology with primary (green) and backup (red) paths. Link delays are shown.

3.1 Components

Component	Role	File
UDP server (h2)	Receives DATA/PROBE datagrams, replies with ACK/PROBE_ACK, detects duplicates, logs arrivals. Receiver thread plus worker threads.	src/server.py
UDP client (h1, h3)	Sends numbered messages at a fixed rate, tracks ACKs, retransmits, detects outages, probes and resends after recovery.	src/client.py
Protocol	16-byte header: type, client id, attempt, sequence number, send timestamp.	src/protocol.py
Ryu controller	Topology discovery, failure detection, shortest-path routing, ARP proxy, port-statistics monitoring, event log. Dynamic mode and static (baseline) mode.	controller/adaptive_router.py
Mininet topology	Five OVS switches, three hosts, links with bandwidth and delay (TCLink).	topology/topo.py
Experiments and analysis	Automated runs with a failure schedule; metrics, tables and graphs.	experiments/, analysis/

3.2 Communication flow

1. A client sends DATA(seq) to the server's IP (10.0.0.2, UDP port 5000); the server answers with ACK(seq).
2. Packets follow flow rules installed by the controller: match on destination IP, output to the next hop of the shortest path.
3. The controller keeps a graph of the network built from LLDP link discovery and switch port-status messages.
4. When a link fails, the controller removes it from the graph, recomputes the paths and overwrites the flow rules, so traffic moves to the backup path.
5. If packets are lost during the switch-over, the client's recovery logic retransmits them.

3.3 Expected network behaviour

Normally traffic from h1 and h3 to h2 uses the primary path s1→s2→s4. If the link s2–s4 fails, the controller should reroute over s1→s3→s5→s4 within milliseconds, and the application should lose few or no messages. The backup path is longer, so the round-trip time rises (measured: about 18 ms on the primary path and about 40 ms on the backup path, Section 8). When the link returns, the controller moves traffic back to the primary path.

4. UDP socket implementation

The application uses plain UDP sockets (socket.SOCK_DGRAM) with no reliability library. Reliability is built at the application layer on purpose.

4.1 Message format

Field	Size	Meaning
type	1 byte	DATA, ACK, PROBE or PROBE_ACK
client id	1 byte	Identifies the client so one server can serve many
attempt	2 bytes	Transmission attempt number (1 = first try)
seq	4 bytes	Per-client sequence number (0 for probes)
send timestamp	8 bytes	Sender time; echoed unchanged in the ACK so the client measures the round-trip time of the exact attempt that was acknowledged

4.2 Server

One receiver thread reads datagrams and puts them on a queue; a pool of worker threads parses them, detects duplicate retransmissions per (client id, sequence number), sends the ACK and logs the arrival time. Malformed datagrams are ignored. The log is written when the server stops (SIGINT/SIGTERM).

4.3 Client and application-level recovery

The client generates one message every 20 ms (50 messages per second by default). It works as a small state machine:

- **NORMAL:** every message is tracked until acknowledged; unacknowledged messages are retransmitted after a 0.15 s timeout, at most 5 times.
- **Outage detection:** if messages are pending and no ACK has arrived for 0.4 s, the client switches to RECOVERING.
- **RECOVERING:** new messages are buffered (no retransmission storm) and a small PROBE datagram is sent every 0.1 s.
- **Back to NORMAL:** the first PROBE_ACK shows the path works again; all unacknowledged and buffered messages are re-sent in order.

A switch (--recovery off) turns the client into a plain fire-and-forget UDP sender, which is used as a baseline in the evaluation.

4.4 Socket-layer test (without the network)

A test script (tests/test_local_recovery.py) places a UDP proxy between client and server that drops all packets for 2 seconds. With recovery enabled no message was lost; with recovery disabled all 100 messages sent during the blackout were lost.

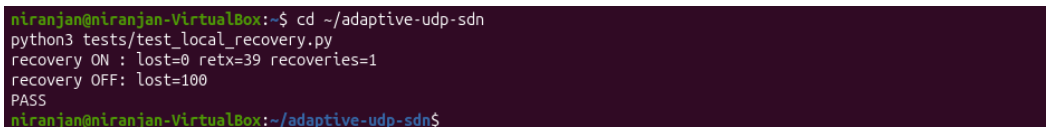


Figure 2. Local socket-layer test: recovery ON loses 0 messages, recovery OFF loses 100.

5. Mininet and SDN implementation

5.1 Topology

The topology (topology/topo.py) has hosts h1 (10.0.0.1), h3 (10.0.0.3) and h2 (10.0.0.2) with fixed MAC addresses, five Open vSwitch switches using OpenFlow 1.3, and TCLink links with bandwidth 10 Mbit/s. The backup path is deliberately slower (the s3–s5 link has 10 ms delay), so the latency change after a failover is visible.

5.2 Controller (Ryu, OpenFlow 1.3)

- **Topology discovery:** LLDP link events (ryu.topology, started with --observe-links) build the link graph.
- **Fast failure detection:** OFPPortStatus (LINK_DOWN) messages from the switches; LLDP timeout is a fallback.
- **Routing:** breadth-first shortest path (hop count) on the current graph; one destination-IP flow per switch and host, at priority 10.
- **ARP proxy:** the topology contains loops, so ARP is answered by the controller and unknown requests are flooded only on host-facing ports, which avoids broadcast storms.
- **Monitoring and logging:** port counters are polled every second (port_stats.csv) and every event is time-stamped (controller_events.csv).
- **Static baseline mode:** with ROUTING_MODE=static the controller installs the same initial paths but ignores link-down events (used for comparison in Section 7).

5.3 Evidence: network up and flow rules installed

After starting the controller and the network, pingall shows full connectivity, and the controller has installed one rule per host on every switch.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

Figure 3. pingall: all three hosts reach each other, 0% dropped.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=493.395s, table=0, n_packets=1076, n_bytes=64560, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=467.381s, table=0, n_packets=12048, n_bytes=698944, priority=10,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=467.381s, table=0, n_packets=12048, n_bytes=1469632, priority=10,ip,nw_dst=10.0.0.2 actions=output:"s1-eth3"
 cookie=0x0, duration=467.381s, table=0, n_packets=4, n_bytes=392, priority=10,ip,nw_dst=10.0.0.3 actions=output:"s1-eth2"
 cookie=0x0, duration=493.943s, table=0, n_packets=75, n_bytes=7256, priority=0 actions=CONTROLLER:65535
```

Figure 4. Flow rules on s1 before the failure: traffic for 10.0.0.2 (the server) leaves via s1-eth3, the port towards s2 (primary path). (Captured in an earlier run of the same topology.)

6. Socket–SDN integration and functional testing

The UDP application and the SDN network were tested together: the client was running and sending to the server while the link s2–s4 was taken down with the Mininet command “link s2 s4 down”.

```
routes_updated reason=host_learned flows=10 took_ms=1.21 paths=[10.0.0.1->10.0.0.2:s1>s2>s4]
host_learned 10.0.0.3 00:00:00:00:00:03 at s1:2
routes_updated reason=host_learned flows=15 took_ms=4.37 paths=[10.0.0.1->10.0.0.2:s1>s2>s4 10.0.0.1->10.0.0.3:s1 10.0.0.2->10.0.0.3:s4>s2>s1]
port_status s4 port=2 down=False
port_status s2 port=2 down=False
port_status s2 port=2 down=True
link_down_detected s2:2 via port_status
routes_updated reason=link_down flows=15 took_ms=5.08 paths=[10.0.0.1->10.0.0.2:s1>s3>s5>s4 10.0.0.1->10.0.0.3:s1 10.0.0.2->10.0.0.3:s4>s5>s3>s1]
port_status s4 port=2 down=True
port_status s4 port=2 down=True
```

Figure 5. Controller log: port_status reports the port down, link_down_detected follows, and routes_updated changes the h1→h2 path from s1>s2>s4 to s1>s3>s5>s4 (5.08 ms to recompute and install all 15 flows).

```
mininet> link s2 s4 down
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=358.944s, table=0, n_packets=758, n_bytes=45480, priority=65535,dl_dst=01:80:c2:00:00:0e,dl_type=0x88cc actions=CONTROLLER:65535
 cookie=0x0, duration=126.287s, table=0, n_packets=12497, n_bytes=725846, priority=10,ip,nw_dst=10.0.0.1 actions=output:"s1-eth1"
 cookie=0x0, duration=126.286s, table=0, n_packets=12629, n_bytes=1533730, priority=10,ip,nw_dst=10.0.0.2 actions=output:"s1-eth4"
 cookie=0x0, duration=126.286s, table=0, n_packets=4, n_bytes=392, priority=10,ip,nw_dst=10.0.0.3 actions=output:"s1-eth2"
 cookie=0x0, duration=358.968s, table=0, n_packets=103, n_bytes=27024, priority=0 actions=CONTROLLER:65535
mininet>
```

Figure 6. Flow rules on s1 after the failure: traffic for 10.0.0.2 now leaves via s1-eth4, the port towards s3 (backup path).

	Before failure	After failure
Path h1 → h2	s1 > s2 > s4	s1 > s3 > s5 > s4
s1 output port for 10.0.0.2	s1-eth3 (towards s2)	s1-eth4 (towards s3)
Controller event	-	port_status, link_down_detected, routes_updated

When the link is brought back up (link s2 s4 up) the controller logs link_up and switches the path back to s1>s2>s4, so both failure and recovery of the network are handled.

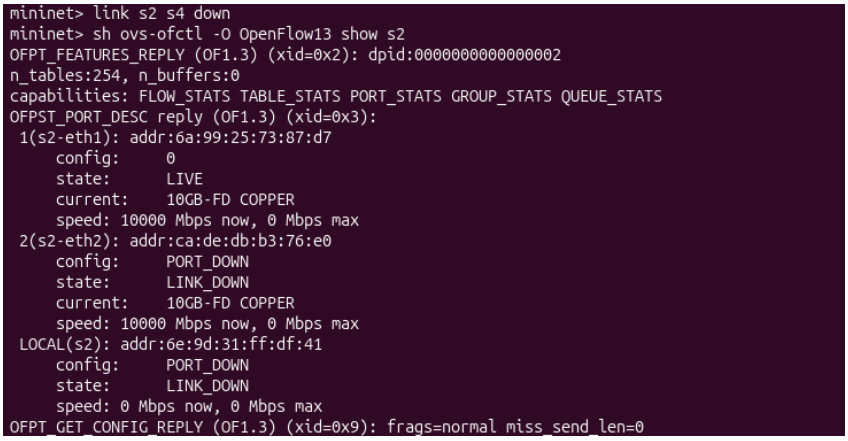


Figure 7. OpenFlow port description of s2 after the failure: port s2-eth2 reports PORT_DOWN / LINK_DOWN (the failure is real, not only logged).

6.1 Functional test cases

ID	Test	Observed result	Evidence
T1	UDP socket layer without network	Recovery ON: 0 lost; OFF: 100 lost; PASS	Figure 2
T2	Connectivity (pingall)	0% dropped (6/6 received)	Figure 3
T3	Controller installs flows	Rule per host on s1; 10.0.0.2 via s1-eth3	Figure 4
T4	Failure detection	link_down_detected via port_status; routes recomputed in 5.08 ms	Figure 5
T5	Reroute to backup path	10.0.0.2 via s1-eth4; path s1>s3>s5>s4	Figures 5, 6
T6	UDP traffic during the failure	4500 sent, 4500 acknowledged, 0 lost, 36 retransmissions	Figure 8
T7	Link restored	Controller returns the path to s1>s2>s4	Controller log

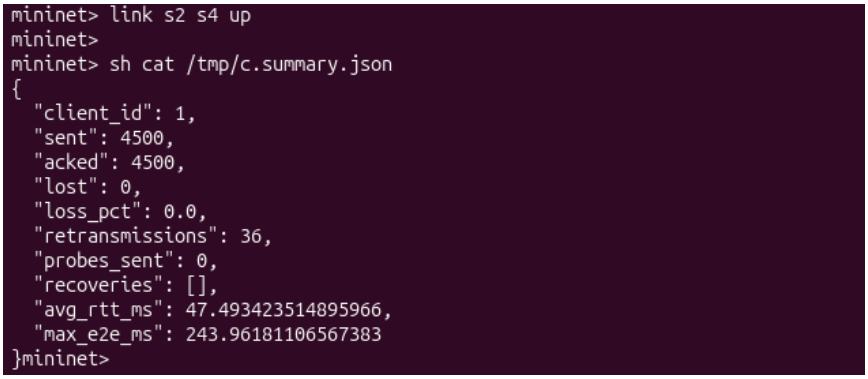


Figure 8. Client summary after a 90-second run (4500 messages) during which the primary link failed and was restored: no message was lost.

Interpretation. All 4500 messages were acknowledged although the primary link failed during the run. The 36 retransmissions include messages sent in the short interval between the failure and the controller's route update, which the client resent over the backup path (a few retransmissions also occur without any failure, for example 17 in an earlier failure-free run, because of queueing delay on the 10 Mbit/s links). The client's outage detector (0.4 s) did not trigger in this run, which suggests the interruption was shorter than 0.4 s, so the SDN reroute alone restored the service quickly. Repeated, measured results follow in Sections 7 to 11.

7. Experimental evaluation setup

7.1 Environment and traffic

All experiments ran on one Ubuntu virtual machine (VirtualBox) with Mininet 2.3.0, Open vSwitch 3.7.1 and Ryu 4.34 (OpenFlow 1.3). Two UDP clients (h1, h3) each send 50 messages per second with a 64-byte payload to the server h2, for 30 s (36 s in the double-failure scenario). Each run is fully automated by experiments/run_experiment.py, which starts the controller, builds the network, runs the server and clients, applies a failure schedule and saves all logs.

7.2 Failure scenarios

Scenario	Failure schedule	Purpose
single	s2-s4 down at 8 s, up at 20 s	Basic failover and recovery
double	s2-s4 down at 8 s; s3-s5 down at 14 s; s3-s5 up at 20 s; s2-s4 up at 26 s	Challenging: no path exists between 14 s and 20 s
flap	s2-s4 down at 8 s, up at 12 s, down at 16 s, up at 20 s	Challenging: repeated failures, stability of the controller
none	no failure	Control: measures the noise of the emulation itself

7.3 Compared configurations

Name	Controller / client	Meaning
static-noapp	static / plain UDP	Baseline: no rerouting, no recovery
static-app	static / recovery	Application recovery only
dynamic-noapp	SDN reroute / plain UDP	SDN rerouting only
dynamic-app	SDN reroute / recovery	Proposed full system

Each configuration was run three times per scenario (the no-failure control only for the baseline and the full system), giving 42 runs. Tables show the mean ± standard deviation over the three runs.

7.4 Metrics

Metric	Definition
Service outage (ms)	Time from the first message that did not get through after the (first) failure to the first later message delivered on its first attempt (client 1). Resolution is the 20 ms message interval.
Loss (%)	Messages never acknowledged, over both clients (application-level loss, after any retransmissions).
Retransmissions	Extra DATA datagrams sent (both clients).
RTT before / on backup (ms)	Mean round-trip time before the failure and while running on the backup path (after the outage).
p95 / max latency (ms)	Time from generating a message to receiving its ACK; includes time a message waits for recovery. Counts delivered messages only.
Detect / reroute / react (ms)	Controller timing: failure injected → link_down_detected; failure injected → new flows pushed; switch reports port down → new flows pushed.

8. Results

8.1 Overall comparison

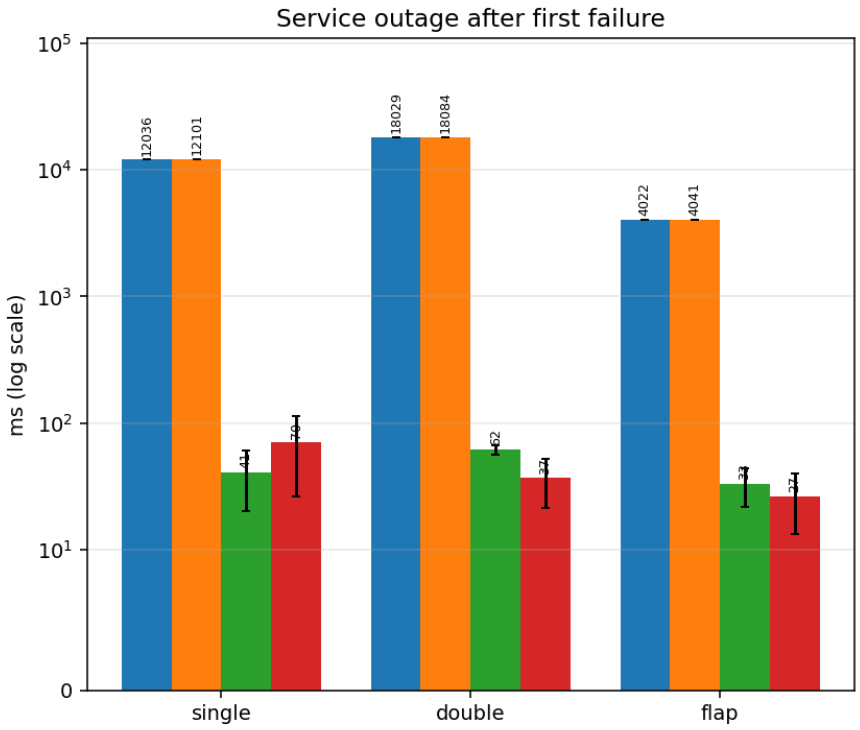


Figure 9. Service outage after the first failure (log scale). Static routing is out for the whole time the link is down; SDN rerouting recovers in tens of milliseconds.

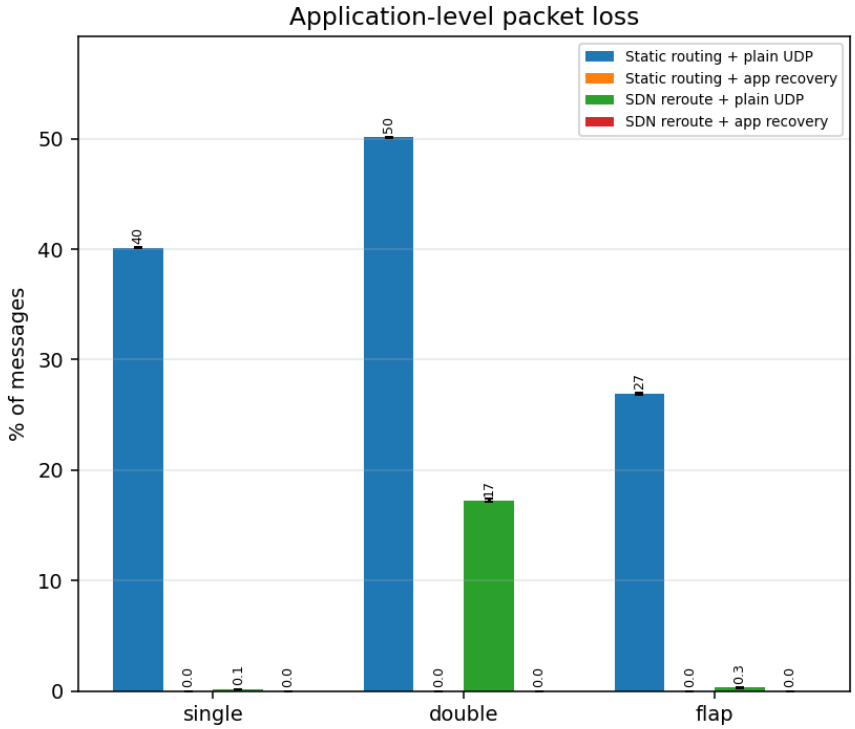


Figure 10. Application-level message loss. Only static plain UDP (and SDN plain UDP when no path exists) lose messages; application recovery gives zero loss in every scenario.

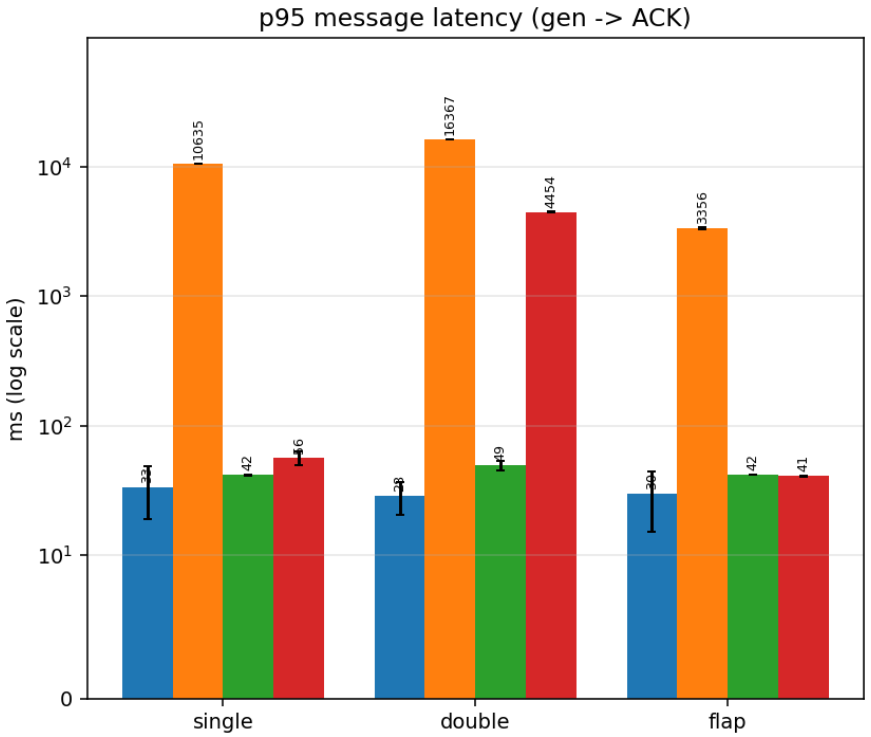


Figure 11. p95 message latency (log scale). Application recovery on static routing delivers every message but up to seconds late. Plain-UDP bars count delivered messages only, so they hide the lost ones.

8.2 Scenario: single link failure

Setup	Outage (ms)	Loss (%)	Retrans.	RTT before / on backup (ms)	p95 latency (ms)	Max latency (ms)
Static routing + plain UDP (baseline)	12036 ± 13	40.1 ± 0.1	0	18.2 / -	33	160
Static routing + app recovery	12101 ± 41	0.0 ± 0.0	77	20.6 / -	10635	12133
SDN reroute + plain UDP	41 ± 21	0.1 ± 0.0	0	17.5 / 39.4	42	82
SDN reroute + app recovery (full system)	**70 ± 44**	**0.0 ± 0.0**	**16**	**23.0 / 44.7**	**56**	**274**

With static routing the traffic is lost for the whole time the link is down: the outage of 12.0 s equals the 12 s the link was down and the baseline loses 40% of its messages (12 s of 30 s). With SDN rerouting the outage is 41 ms (plain UDP) and 70 ms (with recovery); these two values are not significantly different (standard deviations of 21 and 44 ms, n = 3). Loss falls to 0.14% for plain UDP and to 0% with application recovery. Static routing with application recovery also loses nothing, but p95 latency is 10.6 s because messages wait until the link returns.

Round-trip time rises from about 18 ms on the primary path to about 39 ms on the backup path, as expected for the longer, slower route (Figure 12).

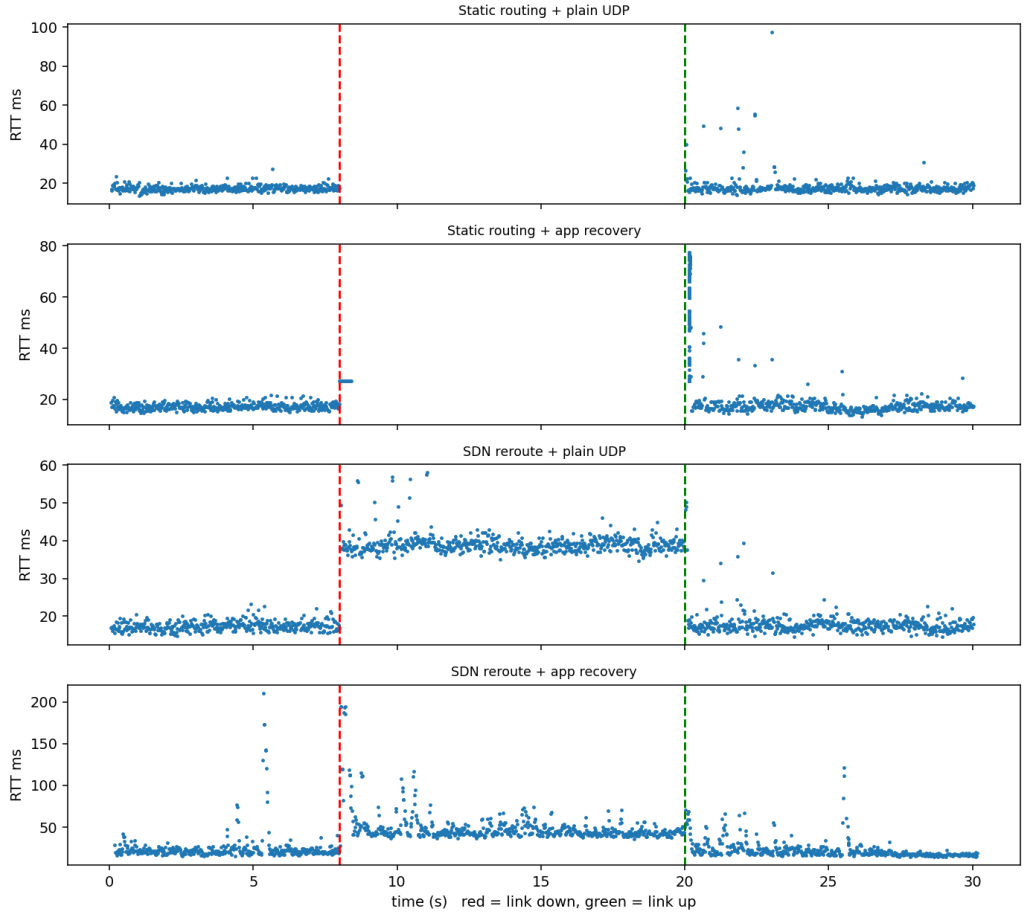


Figure 12. Single failure: RTT over time for the four setups (client 1). Red line = link down, green = link up. Static setups have no data during the outage; SDN setups continue on the slower backup path (about 40 ms) and return to about 18 ms when the link is restored.

8.3 Scenario: double failure (challenging)

Setup	Outage (ms)	Loss (%)	Retrans.	RTT before / on backup (ms)	p95 latency (ms)	Max latency (ms)
Static routing + plain UDP (baseline)	18029 ± 29	50.1 ± 0.1	0	18.7 / -	28	98
Static routing + app recovery	18084 ± 54	0.0 ± 0.0	624	17.6 / -	16367	18146
SDN reroute + plain UDP	62 ± 5	17.3 ± 0.1	0	18.6 / 41.8	49	124
SDN reroute + app recovery (full system)	**37 ± 15**	**0.0 ± 0.0**	**88**	**17.9 / 40.0**	**4454**	**6263**

Between 14 s and 20 s both paths are down, so no routing can deliver data. The controller recognises this: its log marks the server as UNREACHABLE and drop rules are installed. Plain UDP loses 17.3% of messages with SDN (about 6 of 36 s) and 50% with static routing. With application recovery no message is lost in either case; the price is latency. With SDN the p95 latency is 4.5 s and the maximum 6.3 s, roughly the length of the window without any path, because messages are buffered until a path returns. The long maximum gap between server arrivals in this scenario (about 6.1 s for SDN) belongs to this window; the 37–62 ms outage in the table refers to the first failure, which the controller handled by rerouting.

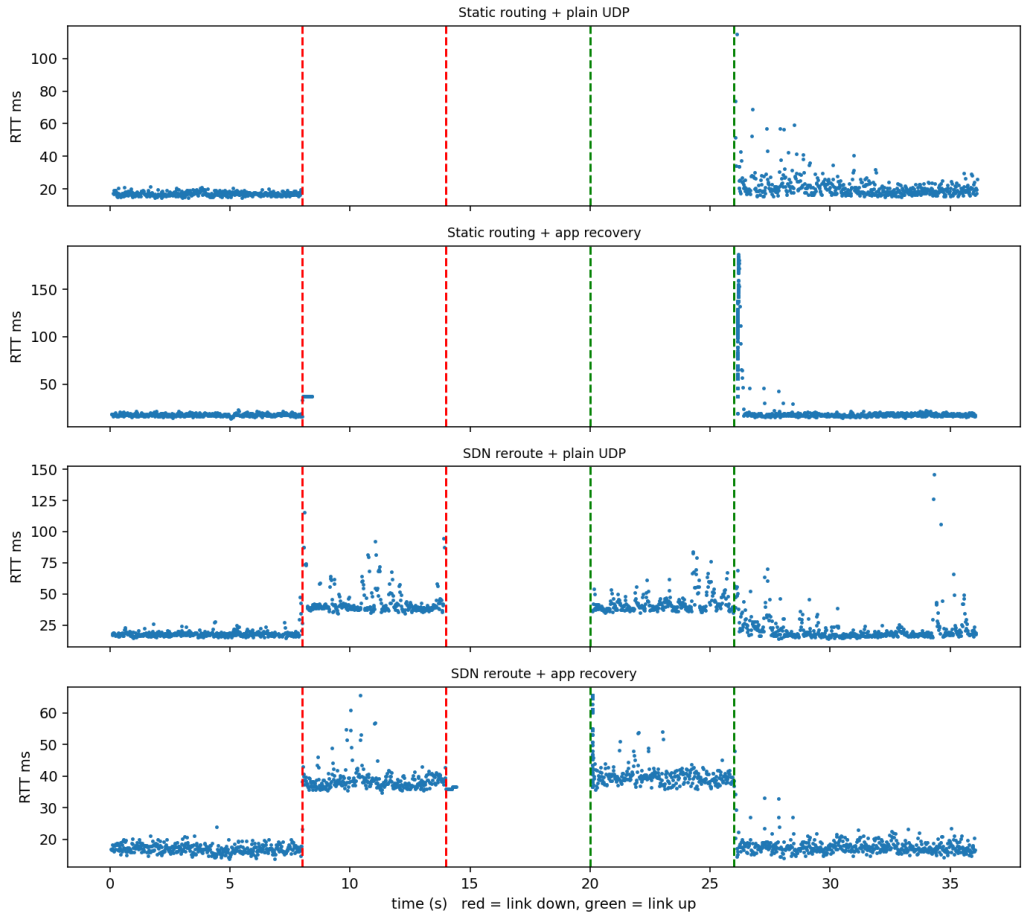


Figure 13. Double failure: static setups are down from 8 s to 26 s. SDN setups run on the backup path from 8 s, have no path from 14 s to 20 s, and recover when s3-s5 returns at 20 s.

8.4 Scenario: link flapping (challenging)

Setup	Outage (ms)	Loss (%)	Retrans.	RTT before / on backup (ms)	p95 latency (ms)	Max latency (ms)
Static routing + plain UDP (baseline)	4022 ± 19	26.9 ± 0.1	0	17.7 / -	30	318
Static routing + app recovery	4041 ± 22	0.0 ± 0.0	156	18.0 / -	3356	4115
SDN reroute + plain UDP	33 ± 11	0.3 ± 0.0	0	18.1 / 40.4	42	49
SDN reroute + app recovery (full system)	**27 ± 13**	**0.0 ± 0.0**	**6**	**17.9 / 39.1**	**41**	**200**

The link fails twice for 4 s each. Static routing is out for 8 s in total (first outage 4.0 s, 26.9% loss). The SDN controller re-converged after every event: outages of 33 ms (plain UDP) and 27 ms (with recovery), loss of 0.3% and 0%. The RTT timeline shows a clean switch between about 18 ms (primary) and about 40 ms (backup) after each event, with no oscillation or instability.

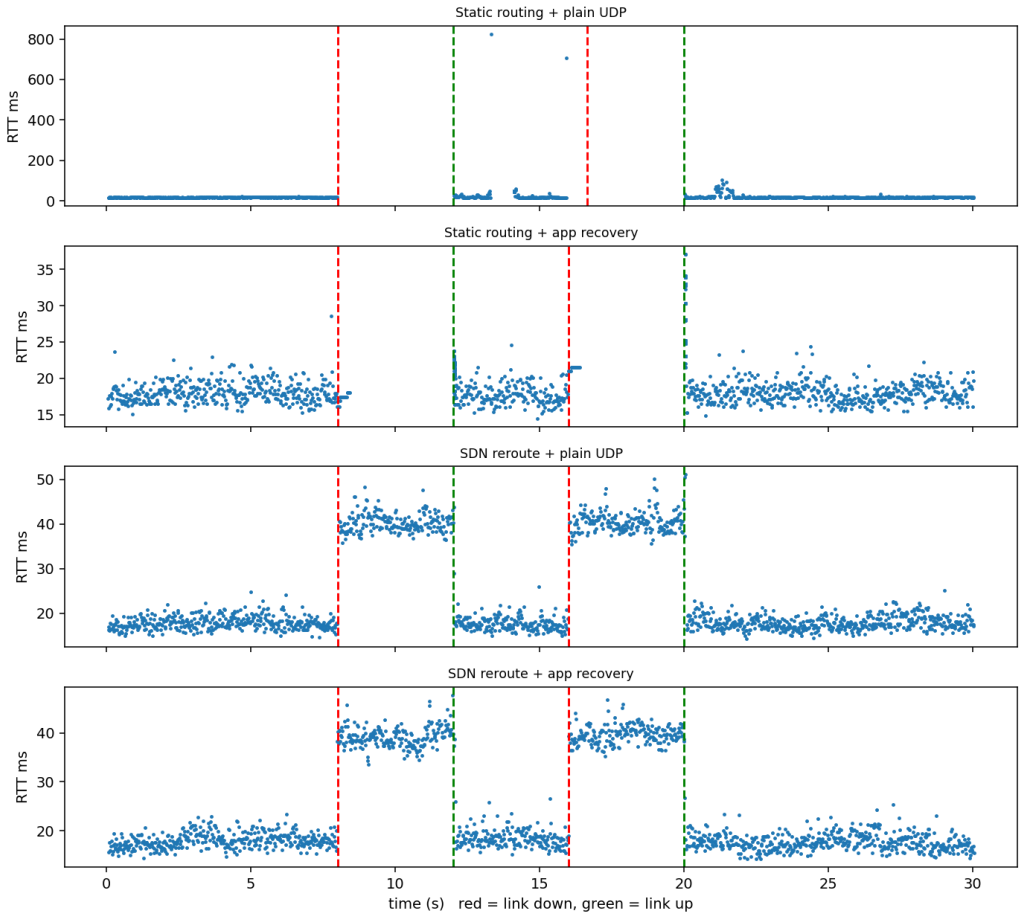


Figure 14. Link flapping: after each down/up event the SDN setups move to the backup or primary path within tens of milliseconds.

8.5 Control run and controller timing

Without any failure, the largest gap between server arrivals is about 28 ms, loss is 0%, and p95 latency is about 21 ms for both setups. This is the noise floor of the emulation: SDN outage values of 26–70 ms are of the same order as it and close to the 20 ms message interval, so they should be read as “tens of milliseconds”, not as exact values.

Scenario / setup	Detect (ms)	Reroute (ms)	React (ms)
single / dynamic-noapp	26 ± 6	30 ± 7	5.2 ± 4.1
single / dynamic-app	37 ± 14	52 ± 34	14.8 ± 21.1
double / dynamic-noapp	34 ± 12	36 ± 13	1.9 ± 0.8
double / dynamic-app	29 ± 3	32 ± 2	3.2 ± 2.6
flap / dynamic-noapp	24 ± 3	31 ± 3	6.7 ± 4.5
flap / dynamic-app	20 ± 4	23 ± 2	3.0 ± 1.8

The controller detects a failure within roughly 20–40 ms of the command to take the link down and has new flows installed a few milliseconds after the switch reports the port down (react time 2–15 ms). Part of the detect time is Mininet itself bringing the interface down.

The numbers above were produced by the analysis script from the raw logs of all 42 runs; Figure 15 shows its output as printed on the VM terminal (the same values are tabulated in this section).

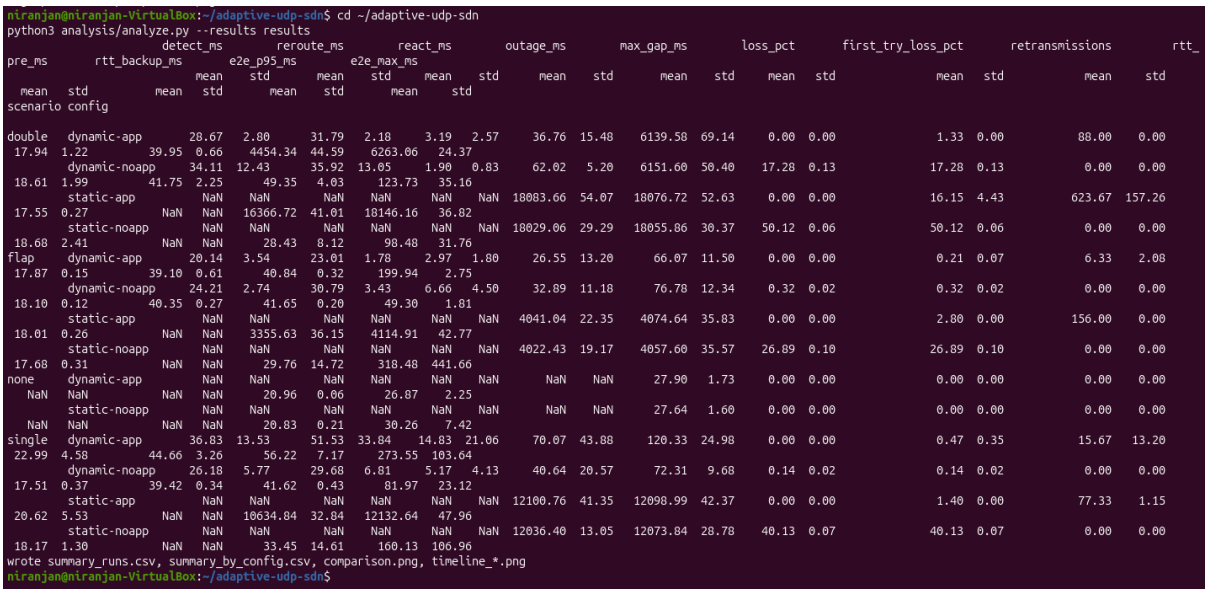


Figure 15. Output of analysis/analyze.py on the 42 experiment runs (terminal screenshot): mean and standard deviation of every metric for each scenario and configuration.

9. Dynamic SDN functionality

The controller (controller/adaptive_router.py) responds to network conditions without any manual action:

- **Topology discovery:** LLDP link events build and update a graph of switches and links.
- **Failure detection:** OpenFlow port-status messages report a failing port within milliseconds; LLDP timeout is the fallback.
- **Path recomputation:** breadth-first shortest path on the current graph for every known host.
- **Rerouting:** new flow rules overwrite the old ones on all switches (15 flows in about 1–6 ms).
- **Recovery:** when a link returns, the controller moves traffic back to the shorter primary path.
- **Unreachable destinations:** if no path exists, a drop rule is installed instead of leaving a stale rule that forwards into a dead link.
- **Monitoring:** per-port counters are polled every second (port_stats.csv) and every event is time-stamped (controller_events.csv).

Example from a controller log (single failure, then double failure):

```
link_down_detected s2:2 via port_status
routes_updated reason=link_down flows=15 took_ms=1.36 paths=[10.0.0.1->10.0.0.2:s1>s3>s5>s4
...]
...
routes_updated reason=link_down flows=15 took_ms=5.79 paths=[10.0.0.1->10.0.0.2:UNREACHABLE
...]
```

10. Challenging scenarios and robustness

- **Double failure (no path):** the controller marks the server unreachable, and the application layer prevents data loss (0% with recovery), at the cost of a latency up to the length of the outage. The system returns to normal after both links are restored.
- **Link flapping:** four failure/recovery events in 12 s are handled one by one with no instability and at most 0.3% loss.
- **Noisy environment:** the VM occasionally shows delay spikes of 100–200 ms even without failures (visible in Figure 12, bottom panel). The control run was used to quantify this noise.

11. Performance comparison with the baseline

Scenario	Baseline outage	Full system outage	Baseline loss → full system
single	12.0 s	70 ms	40.1 % → 0.0 %
double	18.0 s	37 ms	50.1 % → 0.0 %
flap	4.0 s	27 ms	26.9 % → 0.0 %

Compared with static routing, SDN rerouting makes the service outage roughly 150 to 500 times shorter in all three scenarios (for example 12.0 s to 41–70 ms in the single-failure case), and the full system (SDN plus application recovery) delivered every message in every scenario. The two layers complement each other: SDN rerouting makes recovery fast, and application recovery guarantees delivery when the network cannot (double failure) or when a few messages are lost during the switch-over.

12. Limitations

- Only three repetitions per configuration and one VM. The standard deviations are reported; differences of a few tens of milliseconds between the SDN setups (for example 41 ms vs 70 ms in the single scenario) are not statistically meaningful.
- The VM adds jitter (in the control runs the largest gap between arrivals was about 28 ms on average, with occasional delay spikes of 100–200 ms in individual runs). Results from emulation are indicative, not exact for real networks.
- Routing uses hop count and ignores link delay or load; the controller does not do load balancing.
- Outage is measured at the 20 ms message interval, so values below about 40 ms cannot be distinguished.
- Latency statistics count delivered messages only: plain-UDP baselines look fast because lost messages have no latency. Loss and latency must be read together.
- Detection time includes Mininet’s own time to bring an interface down, so it overstates the controller’s reaction; the react time is the better measure of the controller itself.

13. Conclusion

The project shows that an SDN controller can keep a UDP application alive through link failures by detecting the failure from switch events and re-installing shortest-path flows in milliseconds. Against a static-routing baseline the service outage drops from seconds to tens of milliseconds, and with application-level recovery no message is lost even when every path is down for several seconds. Possible extensions are delay-aware or load-aware routing, more repetitions on a less noisy machine, and larger topologies.

14. How to reproduce

Environment: Ubuntu VM (VirtualBox), Mininet 2.3.0, Open vSwitch 3.7.1. Ryu is installed in a separate Python 3.9 environment because the system Python is too new for Ryu.

14.1 Installation

```
sudo apt install mininet openvswitch-switch curl -y
curl -LsSf https://astral.sh/uv/install.sh | sh
source $HOME/.local/bin/env
uv venv --python 3.9 ~/ryu-env
uv pip install --python ~/ryu-env/bin/python "setuptools<68" wheel pbr
uv pip install --python ~/ryu-env/bin/python --no-build-isolation \
    ryu "eventlet==0.30.2" "dnspython<2"
sudo apt install python3-pandas python3-matplotlib python3-numpy -y
```

14.2 Manual demo (two terminals)

Terminal 1: controller

```
cd ~/adaptive-udp-sdn
ROUTING_MODE=dynamic RESULTS_DIR=results/demo \
  ~/ryu-env/bin/ryu-manager --observe-links controller/adaptive_router.py
```

Terminal 2: network and application

```
cd ~/adaptive-udp-sdn
sudo python3 topology/topo.py --cli
mininet> pingall
mininet> h2 python3 src/server.py --log /tmp/s.csv &
mininet> h1 python3 src/client.py --server 10.0.0.2 --duration 90 --log /tmp/c.csv &
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
mininet> link s2 s4 down
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
mininet> link s2 s4 up
mininet> sh cat /tmp/c.summary.json
```

Cleanup after a run: exit the Mininet prompt, run “sudo mn -c”, and stop the controller with Ctrl+C. The socket-layer test runs without Mininet: python3 tests/test_local_recovery.py.

14.3 Automated experiments and analysis

```
cd ~/adaptive-udp-sdn
# one run (about 40 s):
sudo python3 experiments/run_experiment.py --ryu $HOME/ryu-env/bin/ryu-manager \
   --mode dynamic --recovery on --duration 30 \
   --events "8:down:s2-s4,20:up:s2-s4" --out results/single/dynamic-app/run1
# full matrix, 42 runs (about 40 minutes):
sudo env "PATH=$PATH:$HOME/ryu-env/bin" REPS=3 bash experiments/run_all.sh
# tables and graphs:
python3 analysis/analyze.py --results results
```

Outputs: results/summary_by_config.csv, results/summary_runs.csv, results/comparison.png and results/timeline_*.png; every run folder also holds the raw client, server and controller logs.

15. Repository structure

Folder	Files	Purpose
src/	protocol.py, server.py, client.py	UDP application
controller/	adaptive_router.py	Ryu controller (dynamic and static modes)
topology/	topo.py	Mininet topology
experiments/	run_experiment.py, run_all.sh	Automated experiments
analysis/	analyze.py	Metrics, tables and graphs
tests/	test_local_recovery.py	Socket-layer test
results/	summary_*.csv, *.png, run folders	Measured data and graphs
docs/	This report, screenshots	Documentation
README.md	-	Setup and run instructions